Active Learning for Molecular-Docking Algorithm Selection under Heterogeneous Runtime

Sean Wan **Jiabao Brad Wang** **Mustafa Misir**
Duke Kunshan University
Kunshan, China

Abstract

Constructing training data for molecular-docking algorithm selection typically requires evaluating every docking method on every protein–ligand complex. We study selective construction of this instance–algorithm performance matrix through retrospective, cell-wise active learning under heterogeneous runtime. Using MolAS, an existing algorithm-selection framework for molecular docking, we compare Mean-UCB with five runtime-aware acquisition formulations, fitting runtime models only to previously observed training-pool runtimes. Across seven docking methods and five held-out folds, partial acquisition yields decision-useful training data. No evaluated formulation consistently leads across the reported budgets and endpoints: runtime weighting changes algorithm allocation, while matrix-quality diagnostics do not consistently track held-out selection performance. These descriptive findings motivate evaluating acquisition by realized docking cost and downstream decision quality. The study is restricted to one benchmark and does not establish superiority over untested acquisition baselines or performance in prospective docking campaigns.

Keywords: active learning; algorithm selection; molecular docking; protein–ligand interactions; drug discovery; runtime-aware acquisition; structure-based virtual screening.

1 Introduction

AI-driven structure-based drug discovery increasingly orchestrates portfolios of classical and learned docking methods whose performance varies across protein–ligand complexes. Automated algorithm selection (AS) exploits this complementarity by choosing a suitable method for each complex [Rice, 1976, Kerschke et al., 2019], using molecular descriptors, graphs, or pretrained representations [Chen et al., 2023, Cao et al., 2026, Wang et al., 2026]. Training a selector, however, ordinarily requires executing every docking method on every training complex to construct a complete performance matrix. We instead formulate this process as resource-constrained computational experimental design: active learning (AL) selects a complex–algorithm cell, acquires its docking outcome, updates the matrix model, and repeats [Cohn et al., 1996, Tharwat and Schenck, 2023, Huseljic et al., 2026].

This setting differs from docking AL under a fixed workflow [Graff et al., 2021, Marin et al., 2024]: each query identifies both the protein–ligand complex and the docking method, and the resulting partial matrix must support decisions on unseen complexes. Our technical basis is active matrix completion [Mısır, 2019]. Previous studies explored uncertainty-guided and runtime-aware acquisitions for AS in combinatorial optimization [Volpato and Song, 2019, Kus et al., 2026]; here, we investigate cell-wise acquisition for molecular-docking AS.

RQ1. To what extent can cell-wise AL reduce molecular-docking evaluations while preserving held-out MolAS decision quality?

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: AI for Drug Discovery: Bridging the Translation Gap.

Query count alone is insufficient because docking runtime varies by complex and algorithm and is unknown before execution. Cost-blind AL may select informative but slow runs, whereas utility-per-cost rules may defer expensive yet selector-relevant evidence.

RQ2. How do alternative predicted-runtime transformations change algorithm allocation and the downstream performance–cost frontier?

We empirically evaluate cell-wise acquisition on MolAS [Wang et al., 2026] through historical replay, comparing Mean-UCB with five predicted-runtime acquisition-score formulations. Our results address a translation challenge in AI for drug discovery: partial acquisition can support useful selectors, but the effect of cost awareness depends on formulation and budget, and computational proxies do not consistently track downstream molecular decisions. To support reproducibility, we release the processed benchmark dataset on Hugging Face[1] and the source code and experimental artifacts on GitHub[2] as publicly accessible resources.

2 Background

Active performance acquisition for algorithm selection. Standard AL generally treats an instance-level label as the query unit [Tharwat and Schenck, 2023], whereas AS admits instance–algorithm outcomes. Active matrix completion (AMC) selects entries of an incomplete performance matrix to reveal [Chakraborty et al., 2013], and supervised AMC can incorporate acquisition cost [Huang et al., 2018]. AMC has been applied directly to AS [Mısır, 2019]; related work studies instance-level active acquisition for time-expensive AS [Volpato and Song, 2019], while Frugal AS combines uncertainty-based acquisition with learned timeout models [Kus et al., 2026]. These studies motivate selective acquisition under limited computational budgets. Recent AL work also highlights the influence of pool management and evaluation protocols on observed performance [Huseljic et al., 2026]. Low reconstruction error need not preserve algorithm rankings or downstream molecular decisions; MolAS provides a structure-based virtual-screening backend for evaluating this distinction [Wang et al., 2026].

Cost-aware acquisition under heterogeneous runtime. Cost-sensitive AL trades informativeness against annotation or execution cost [Tomanek and Hahn, 2010, Gao and Saar-Tsechansky, 2020]. Return-on-investment rules rank queries by estimated benefit per unit cost [Haertel et al., 2015]; more generally, U/C^{γ} interpolates between cost-blind utility ($\gamma = 0$) and direct utility per cost ($\gamma = 1$). Pareto analyses characterize utility–cost trade-offs [Guinet et al., 2020], while budgeted Bayesian optimization studies non-myopic acquisition under unknown evaluation costs [Astudillo et al., 2021]. Batch acquisition adds potential redundancy among individually attractive queries, motivating joint strategies such as BatchBALD [Kirsch et al., 2019]. In molecular docking, favoring inexpensive cells can systematically reshape which algorithms contribute training outcomes. We therefore evaluate runtime-aware acquisition through realized cost, allocation, and held-out MolAS performance, alongside matrix diagnostics.

3 Methodology

3.1 Closed-loop active acquisition of AS training data

Let $\mathcal{I} = \{i_1, \ldots, i_n\}$ denote training-pool protein–ligand complexes and $\mathcal{A} = \{a_1, \ldots, a_m\}$ a portfolio of m docking algorithms. Their strict performance and acquisition cost matrices are

$$\mathbf{Y} \in [0,1]^{n \times m}, \qquad \mathbf{C} \in \mathbb{R}_{+}^{n \times m}. \tag{1}$$

In our retrospective evaluation, querying $q = (i, a)$ reveals its recorded docking score and runtime, (Y_{ia}, C_{ia}). At round t, only cells in Ω_t are observed; the remaining pool is $\mathcal{U}_t = (\mathcal{I} \times \mathcal{A}) \setminus \Omega_t$. Paired trajectories share a runtime-blind 1% warm start, distributed as evenly as possible across algorithms.

[1] https://huggingface.co/datasets/SeanWan05/MolAS-Active-Learning

[2] https://github.com/SeanWan514/MolAS_Active_Learning

Given Ω_t, iterative low-rank SVD completion [Mazumder et al., 2010] estimates unobserved performance. An ensemble of ranks $\mathcal{K} = \{1, 2, 3, 5\}$ produces

$$\mu_{ia}^{Y} = \frac{1}{|\mathcal{K}|} \sum_{k \in \mathcal{K}} \widehat{Y}_{ia}^{(k)}, \qquad \sigma_{ia}^{Y} = \mathrm{SD}_{k \in \mathcal{K}} \big(\widehat{Y}_{ia}^{(k)} \big). \tag{2}$$

These ranks provide different low-rank hypotheses for the seven-column matrix. An acquisition rule assigns $\alpha_t(i, a)$ to each unqueried cell. Selected cells are revealed, and the completion and runtime models are updated. Scores are reranked after batches of at most 50 queries, truncated at checkpoint boundaries; this reduces refitting frequency while holding scores fixed within a batch. A completed training matrix is exported at every integer acquisition percentage from 1% to 100%. No new docking jobs are executed.

We evaluate 3,179 Binding-MOAD complexes [Hu et al., 2005], as in MolAS [Wang et al., 2026], with seven algorithms: Smina [Koes et al., 2013], AutoDock Vina [Trott and Olson, 2010], DiffDock and DiffDock-L [Corso et al., 2023, 2024], SurfDock [Cao et al., 2025], GNINA [McNutt et al., 2021], and Uni-Mol Docking V2 [Alcaide et al., 2026]. MolAS combines RMSD with a PoseBusters-validity gate [Buttenschoen et al., 2024]. We retain the original five outer folds. At each evaluated acquisition checkpoint, MolAS is trained on the completed matrix for that fold's training complexes: queried cells retain observed scores, while unqueried cells receive SVD estimates. Completion and supervised model fitting use the 80% training split; held-out outcomes are used only for evaluation. The downstream comparison uses the seed-0 acquisition archive with matched warm starts; summaries are over outer folds, not independent acquisition/selector-seed replications.

We report valid-pose success below 1 and 2 Å, an AS composite score, and closure of the single-best-solver (SBS) to virtual-best-solver (VBS) gap. Matrix MAE, row-wise Spearman ρ, pairwise concordance, NDCG@3, and decision regret are diagnostics. Reported cost sums recorded docking-job runtimes, including the warm start, and excludes feature/model-fitting overhead and parallel scheduling. Fold summaries are descriptive, without confidence intervals or significance tests; overlapping training splits are not independent replications.

3.2 Mean-UCB acquisition utility

To examine runtime handling within a common acquisition framework, we use Mean-UCB throughout the comparison. Adapted from Qin et al. [2025], it combines predicted performance with ensemble dispersion:

$$U_t(i, a) = \mu_{ia}^{Y} + \beta \sigma_{ia}^{Y}. \tag{3}$$

We fix $\beta = 1$ and the predictive model class across policies; each policy updates its predictions from its own acquired observations. Dispersion across SVD ranks is a heuristic uncertainty measure, not a calibrated confidence bound.

3.3 Runtime-aware acquisition from observed costs

Unqueried runtime is predicted from information available before revealing that cell:

$$\widehat{C}_{ia} = h_t(\mathbf{x}_i, \mathbf{d}_i, \mathbf{e}_a), \tag{4}$$

where $\mathbf{x}_i$ is a fixed 256-dimensional projection of normalized protein–ligand representations, $\mathbf{d}_i$ contains four physical descriptors, and $\mathbf{e}_a$ is a one-hot algorithm identifier. A two-hidden-layer MLP with widths $(32, 16)$ is trained on $\log(1 + C_{ia})$ using only runtimes revealed in Ω_t. Predictions are transformed back to seconds and clipped to the observed-runtime 5th–95th percentile bounds $[\ell_t, u_t]$, with numerical safeguards. Clipping limits extreme predictions but can also underrepresent runtime tails. Unqueried runtimes and full-matrix runtime statistics are excluded from acquisition. Ranks, reranking interval, β, and clipping define the evaluated configuration; their sensitivity has not been established.

All utility ranks and cost normalizations use the current unqueried training pool $\mathcal{U}_t$, across algorithms, separately for each policy. Let $\widetilde{U}_t = \max\{10^{-3}, \mathrm{rank}_{\mathrm{avg}}(U_t)/|\mathcal{U}_t|\}$, using ascending ranks with averaged ties. For a candidate-cost vector c, define

$$\mathcal{S}_t[c](q) = 1 + 4 \frac{c(q) - \min_{r \in \mathcal{U}_t} c(r)}{\max_{r \in \mathcal{U}_t} c(r) - \min_{r \in \mathcal{U}_t} c(r)}. \tag{5}$$

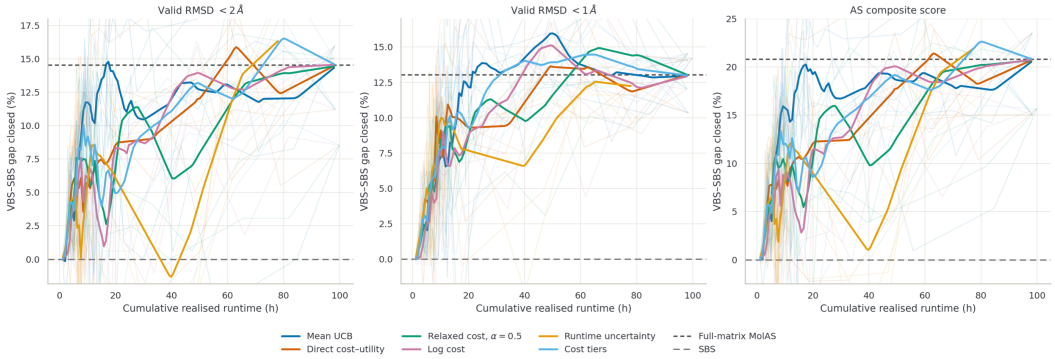


Figure 1: Held-out MolAS SBS–VBS gap closure versus cumulative realized runtime. Bold curves show descriptive five-fold summaries; light curves show fold trajectories. Dashed lines denote SBS and full-matrix MolAS.

If the extrema are numerically equal, $\mathcal{S}_t[c] = 1$. Set $\widetilde{C} = \mathcal{S}_t[\widehat{C}]$. With cell indices suppressed, we compare

$$\alpha_{\text{plain}} = U_t, \qquad \alpha_{\text{direct}} = \frac{U_t}{\widehat{C} + \epsilon}, \qquad \alpha_{\text{sqrt}} = \frac{\widetilde{U}_t}{\sqrt{\widetilde{C}}},$$
$$\alpha_{\text{log}} = \frac{\widetilde{U}_t}{\log(1 + \widetilde{C})}, \qquad \alpha_{\text{tier}} = \frac{\widetilde{U}_t}{g_5(\widehat{C})}, \qquad \alpha_{\text{unc}} = \frac{\widetilde{U}_t}{\sqrt{\widetilde{C}^+}}. \tag{6}$$

Here $\epsilon = 10^{-6}$, and g_5 assigns costs to five candidate-pool quantile tiers, with boundary ties assigned to the lower tier. For runtime uncertainty, three predictors are individually transformed and clipped; their mean μ^C and population SD σ^C give $C^+ = \text{clip}(\mu^C + \sigma^C, \ell_t, u_t)$ and $\widetilde{C}^+ = \mathcal{S}_t[C^+]$. Other cost-aware policies use one predictor. The mean-plus-SD penalty is heuristic, not a calibrated runtime bound.

Direct cost-utility uses raw U_t and predicted seconds; the other cost-aware variants use percentile utility and transformed costs. We therefore compare complete acquisition-score formulations sharing a Mean-UCB definition, rather than isolating the denominator alone. The acquisition logarithm is distinct from the log-runtime regression target.

4 Results

4.1 Partial active acquisition produces decision-useful AS data

Figure 1 reports held-out SBS–VBS gap closure against cumulative realized docking runtime. Across the three endpoints, acquisition recovers substantial decision quality before exhaustive matrix completion. Plain Mean-UCB is competitive at low and intermediate realized cost, while runtime-aware variants exhibit different trajectories for valid RMSD below 1 Å, valid RMSD below 2 Å, and the AS composite score.

No evaluated policy leads across all endpoints and budget regions. Some partial-matrix checkpoints match or exceed full-matrix MolAS, although fold trajectories vary substantially, particularly at small budgets. These comparisons do not establish that omitting labels improves generalization.

4.2 Cost awareness changes the frontier, but not uniformly

Despite sharing a Mean-UCB definition, folds, warm starts, completion model, and update schedule, the formulations yield different performance–cost trajectories. Because utility and cost scalings differ, denominator transformations alone do not establish an ordering of cost pressure. None consistently leads across the reported endpoints and budgets. Runtime-aware policies bring inexpensive algorithms forward and defer some expensive queries (Appendix A.2); greater acquisition volume does not consistently yield better held-out performance.

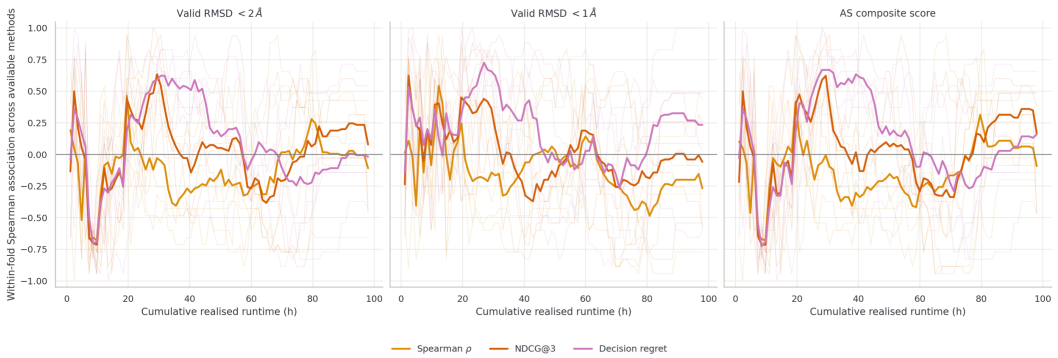


Figure 2: Within-fold Spearman associations across available acquisition policies between matrix diagnostics and held-out MolAS gap closure over realized runtime. Colors identify diagnostics; light curves show fold-level associations and bold curves summarize their trends. Decision regret is sign-reversed before computing correlations.

Figure 2 shows that associations between matrix diagnostics and held-out MolAS gap closure vary with acquisition stage and endpoint. Sign-reversed decision regret exhibits a positive association over several intermediate-cost regions, particularly for valid RMSD below 1 Å, whereas row-wise Spearman ρ is frequently weak or negative. NDCG@3 is informative in some regions but unstable across endpoints. Fold-level trajectories also vary, especially early in acquisition. No evaluated diagnostic consistently tracks downstream quality across this comparison.

Runtime prediction adds another source of uncertainty. Appendix A.1 documents early under-prediction for Uni-Mol and both DiffDock variants, and a long positive error tail for AutoDock Vina. Realized runtime, allocation, matrix diagnostics, and held-out quality provide complementary evidence; these associations do not isolate the causal contribution of prediction errors.

5 Discussion, Limitations, and Conclusion

Discussion. Runtime-aware acquisition changes both how many cells can be queried within a budget and which docking algorithms contribute evidence. In the evaluated portfolio, cost-aware policies bring inexpensive algorithms forward and defer some expensive queries, increasing acquisition volume without consistently improving held-out MolAS quality. Bridging computational benchmarks to drug-discovery decisions requires evaluation by realized runtime, allocation, matrix fidelity, and held-out molecular decision quality. Reconstruction diagnostics remain useful for assessing imputation but are insufficient proxies for selector quality.

For practical use, specify the endpoint and budget in advance; choose acquisition and stopping settings on separate validation complexes, counting their labeling cost, and reserve test folds for final evaluation. Monitor per-engine runtime errors and acquisition coverage alongside selector quality. These recommendations are not a validated deployment procedure.

Limitations and Future Work. We study one curated benchmark and seven algorithms, using SVD completion and myopic Mean-UCB formulations. Random-cell, pure-uncertainty, and joint-batch baselines are absent; completion-induced bias is not separately assessed. Runtime is hardware-dependent; our sequential cost model does not address parallel scheduling, failures, or timeouts. Unqueried outcomes remain hidden during acquisition. Broader baselines, completion models, statistical characterization, sensitivity studies, runtime calibration, additional datasets/portfolios, and prospective validation are detailed in Appendix A.3.

Conclusion. Partial docking evaluations can construct decision-useful AS training data. No evaluated formulation consistently leads across the reported endpoints and budgets. Cost awareness changes both computation and the evidence available to the selector. Observed-runtime fitting, realized-cost reporting, allocation audits, and held-out MolAS evaluation together provide an empirical reference for runtime-aware acquisition in molecular docking.

References

Eric Alcaide, Zhifeng Gao, Guolin Ke, et al. Uni-Mol Docking V2: Towards realistic and accurate binding pose prediction. In *Artificial Neural Networks and Machine Learning. ICANN 2025 International Workshops and Special Sessions*, pages 34–41. Springer, 2026. doi: 10.1007/978-3-032-04552-2_5.

Raul Astudillo, Daniel Jiang, Maximilian Balandat, Eytan Bakshy, and Peter I. Frazier. Multi-step budgeted bayesian optimization with unknown evaluation costs. In *Advances in Neural Information Processing Systems*, volume 34, pages 20197–20209, 2021.

Martin Buttenschoen, Garrett M. Morris, and Charlotte M. Deane. PoseBusters: Ai-based docking methods fail to generate physically valid poses or generalise to novel sequences. *Chemical Science*, 15:3130–3139, 2024. doi: 10.1039/D3SC04185A.

Duanhua Cao et al. SurfDock is a surface-informed diffusion generative model for reliable and accurate protein–ligand complex prediction. *Nature Methods*, 22(2):310–322, 2025.

Siyuan Cao, Hongxuan Wu, Jiabao Brad Wang, Yiliang Yuan, and Mustafa Mısır. MC-GNNAS-Dock: Multi-criteria GNN-based algorithm selection for molecular docking. In *PRICAI 2025: Trends in Artificial Intelligence*, pages 686–696. Springer, 2026. doi: 10.1007/978-981-95-7075-1_46.

Shayok Chakraborty, Jiayu Zhou, Vineeth Balasubramanian, Sethuraman Panchanathan, Ian Davidson, and Jieping Ye. Active matrix completion. In *2013 IEEE 13th International Conference on Data Mining*, pages 81–90, 2013. doi: 10.1109/ICDM.2013.69.

Tianlai Chen, Xiwen Shu, Huiyuan Zhou, Floyd A. Beckford, and Mustafa Mısır. Algorithm selection for protein–ligand docking: Strategies and analysis on ACE. *Scientific Reports*, 13(1):8219, 2023. doi: 10.1038/s41598-023-35132-5.

David A. Cohn, Zoubin Ghahramani, and Michael I. Jordan. Active learning with statistical models. *Journal of Artificial Intelligence Research*, 4:129–145, 1996. doi: 10.1613/jair.295.

Gabriele Corso, Hannes Stärk, Bowen Jing, Regina Barzilay, and Tommi Jaakkola. DiffDock: Diffusion steps, twists, and turns for molecular docking. In *International Conference on Learning Representations*, 2023. URL `https://openreview.net/forum?id=kKF8_K-mBbS`.

Gabriele Corso, Arthur Deng, Benjamin Fry, Nicholas Polizzi, Regina Barzilay, and Tommi Jaakkola. Deep confident steps to new pockets: Strategies for docking generalization. *arXiv preprint arXiv:2402.18396*, 2024. doi: 10.48550/arXiv.2402.18396.

Ruijiang Gao and Maytal Saar-Tsechansky. Cost-accuracy aware adaptive labeling for active learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 2569–2576, 2020. doi: 10.1609/aaai.v34i03.5640.

David E. Graff, Eugene I. Shakhnovich, and Connor W. Coley. Accelerating high-throughput virtual screening through molecular pool-based active learning. *Chemical Science*, 12(22):7866–7881, 2021. doi: 10.1039/D0SC06805E.

Gauthier Guinet, Valerio Perrone, and Cédric Archambeau. Pareto-efficient acquisition functions for cost-aware bayesian optimization. *arXiv preprint arXiv:2011.11456*, 2020. doi: 10.48550/arXiv.2011.11456.

Robbie Haertel, Eric Ringger, Kevin Seppi, and Paul Felt. An analytic and empirical evaluation of return-on-investment-based active learning. In *Proceedings of the 9th Linguistic Annotation Workshop*, pages 11–20. Association for Computational Linguistics, 2015. doi: 10.3115/v1/W15-1602.

Liegi Hu, Mark L. Benson, Richard D. Smith, Michael G. Lerner, and Heather A. Carlson. Binding MOAD (mother of all databases). *Proteins: Structure, Function, and Bioinformatics*, 60(3):333–340, 2005. doi: 10.1002/prot.20512.

Sheng-Jun Huang, Miao Xu, Ming-Kun Xie, Masashi Sugiyama, Gang Niu, and Songcan Chen. Active feature acquisition with supervised matrix completion. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1571–1579. Association for Computing Machinery, 2018. doi: 10.1145/3219819.3220084.

Denis Huseljic, Marek Herde, Lukas Rauch, Paul Hahn, and Bernhard Sick. Cleaning the pool: Progressive filtering of unlabeled pools in deep active learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 22238–22247, June 2026.

Pascal Kerschke, Holger H. Hoos, Frank Neumann, and Heike Trautmann. Automated algorithm selection: Survey and perspectives. *Evolutionary Computation*, 27(1):3–45, 2019. doi: 10.1162/evco_a_00242.

Andreas Kirsch, Joost van Amersfoort, and Yarin Gal. BatchBALD: Efficient and diverse batch acquisition for deep bayesian active learning. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

David Ryan Koes, Matthew P. Baumgartner, and Carlos J. Camacho. Lessons learned in empirical scoring with smina from the CSAR 2011 benchmarking exercise. *Journal of Chemical Information and Modeling*, 53(8):1893–1904, 2013. doi: 10.1021/ci300604z.

Erdem Kus, Özgür Akgün, Nguyen Dang, Ian Miguel, and Lars Kotthoff. Frugal algorithm selection for combinatorial search. *Journal of Artificial Intelligence Research*, 86, 2026.

Egor Marin, Margarita Kovaleva, Maria Kadukova, Khalid Mustafin, Polina Khorn, Andrey Rogachev, Alexey Mishin, Albert Guskov, and Valentin Borshchevskiy. Regression-based active learning for accessible acceleration of ultra-large library docking. *Journal of Chemical Information and Modeling*, 64(7):2612–2623, 2024. doi: 10.1021/acs.jcim.3c01661.

Rahul Mazumder, Trevor Hastie, and Robert Tibshirani. Spectral regularization algorithms for learning large incomplete matrices. *Journal of Machine Learning Research*, 11:2287–2322, 2010. URL `https://www.jmlr.org/papers/v11/mazumder10a.html`.

Andrew T. McNutt, Paul Francoeur, Rishal Aggarwal, Tomohide Masuda, Rocco Meli, Matthew Ragoza, Jocelyn Sunseri, and David Ryan Koes. GNINA 1.0: Molecular docking with deep learning. *Journal of Cheminformatics*, 13(1):43, 2021. doi: 10.1186/s13321-021-00522-2.

Mustafa Mısır. Active matrix completion for algorithm selection. In *Machine Learning, Optimization, and Data Science*, pages 321–334. Springer, 2019. doi: 10.1007/978-3-030-37599-7_27.

Jason Qin, Hans-Hermann Wessels, Carlos Fernandez-Granda, and Yuhan Hao. Active learning for efficient discovery of optimal combinatorial perturbations. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 50363–50383. PMLR, 2025. URL `https://proceedings.mlr.press/v267/qin25g.html`.

John R. Rice. The algorithm selection problem. In *Advances in Computers*, volume 15, pages 65–118. Elsevier, 1976. doi: 10.1016/S0065-2458(08)60520-3.

Alaa Tharwat and Wolfram Schenck. A survey on active learning: State-of-the-art, practical challenges and research directions. *Mathematics*, 11(4):820, 2023. doi: 10.3390/math11040820.

Katrin Tomanek and Udo Hahn. A comparison of models for cost-sensitive active learning. In *COLING 2010: Posters*, pages 1247–1255. Coling 2010 Organizing Committee, 2010. URL `https://aclanthology.org/C10-2143/`.

Oleg Trott and Arthur J. Olson. AutoDock Vina: Improving the speed and accuracy of docking with a new scoring function, efficient optimization, and multithreading. *Journal of Computational Chemistry*, 31(2):455–461, 2010. doi: 10.1002/jcc.21334.

Riccardo Volpato and Guangyan Song. Active learning to optimise time-expensive algorithm selection. *arXiv preprint arXiv:1909.03261*, 2019.

Jiabao Brad Wang, Siyuan Cao, Hongxuan Wu, Yiliang Yuan, and Mustafa Mısır. Molecular embedding-based algorithm selection in protein–ligand docking. *Journal of Cheminformatics*, 18 (1):47, 2026.

A Runtime heterogeneity and the mechanism of cost-aware acquisition

A.1 Runtime scale and recoverability

Algorithm identity provides a strong first-order cost signal (Figure 3), but a single average does not describe the observed within-engine tails. Uni-Mol and the two DiffDock variants are consistently costly, whereas Smina and SurfDock are usually cheap. AutoDock Vina is a consequential exception: its median is below those of Uni-Mol and the DiffDock variants, but its upper tail spans orders of magnitude. Cost-aware acquisition therefore encounters both between-algorithm scale differences and within-algorithm tail risk. AutoDock Vina has a median runtime of 12 s but a mean of 48.4 s and accounts for 42.7 of the 123.2 hours required by the full matrix.

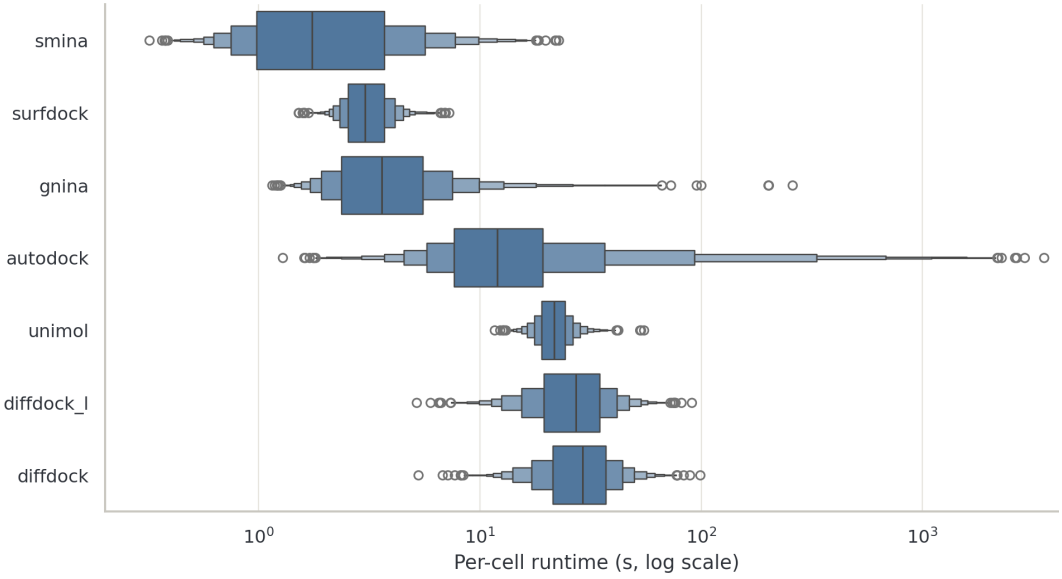


Figure 3: Per-cell runtime distributions across 3,179 MOAD-curated complexes.

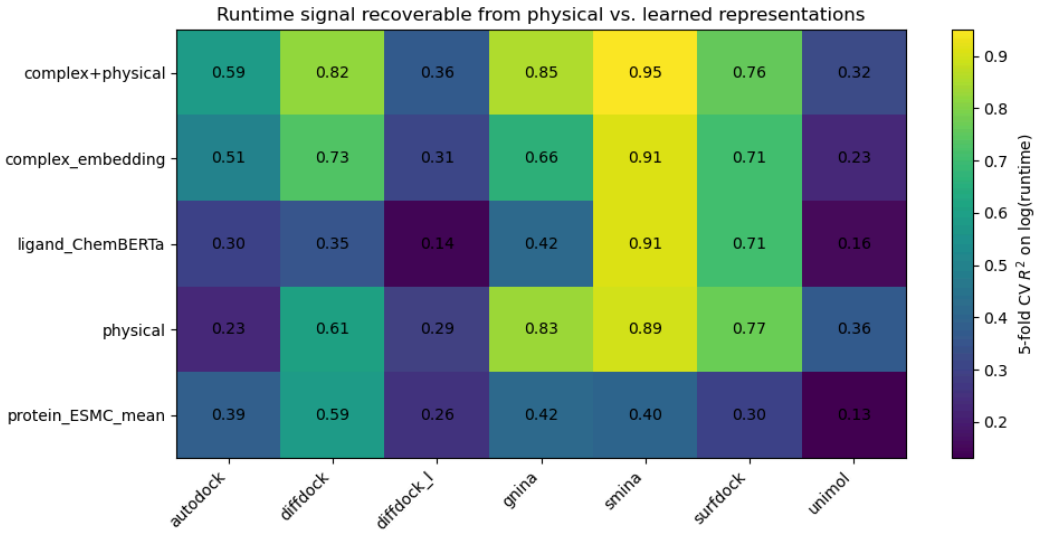


Figure 4: Mean five-fold cross-validation R^2 from engine-specific ridge probes on log runtime.

The ridge probes in Figure 4 show that runtime is linearly predictable from the available features for some engines under this diagnostic protocol. The combined complex representation and physical

descriptors explain log runtime well for Smina, GNINA, DiffDock, and SurfDock, but only moderately for AutoDock Vina and weakly for DiffDock-L and Uni-Mol. Adding representations is not uniformly beneficial. A low probe R^2 does not establish that nonlinear prediction is impossible. Conversely, probe performance does not validate the calibration of the acquisition-time MLP trained on selectively observed runtimes.

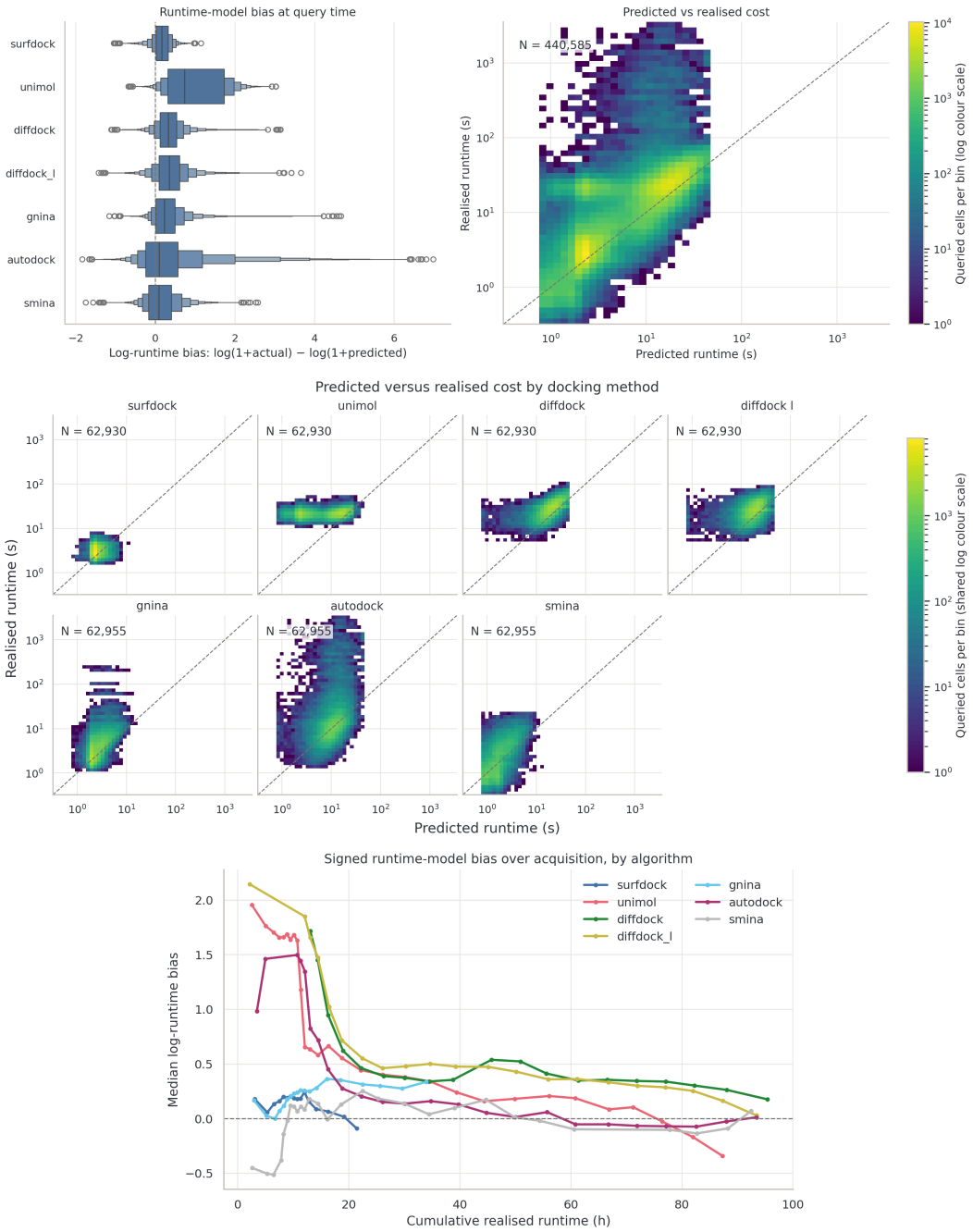


Figure 5: Post-warm-start online runtime diagnostics for 440,585 queried-cell records pooled over five cost-aware policies and five folds. Top left: per-algorithm signed log-runtime error; positive values denote underprediction. Top right: pooled density of realized versus predicted runtime. Middle: per-algorithm densities. Bottom: within-bin median signed error over cumulative realized cost. Records can repeat historical cells across policies and overlapping folds.

For a queried cell, define the signed error $\log(1 + C) - \log(1 + \widehat{C})$, so positive values denote underprediction. In Figure 5, excluding warm-start observations, the pooled queried-cell distribution is shifted above zero, although errors differ by engine. Uni-Mol and both DiffDock variants are strongly underpredicted initially, whereas Smina is initially overpredicted; several trajectories approach or cross zero later. AutoDock Vina has a less extreme central bias than Uni-Mol but a longer positive tail, corresponding to rare runtimes far beyond its central predictions. The bottom panel pools policies and folds before forming cumulative-cost bins and reports the within-bin median for each engine.

These records describe acquired cells, not calibration over the unqueried pool. The same historical docking outcome can appear in multiple policies and overlapping training folds, so records are not independent docking measurements. The plots document systematic error but neither identify its cause nor evaluate a correction.

A.2 Effect of cost awareness on acquisition allocation

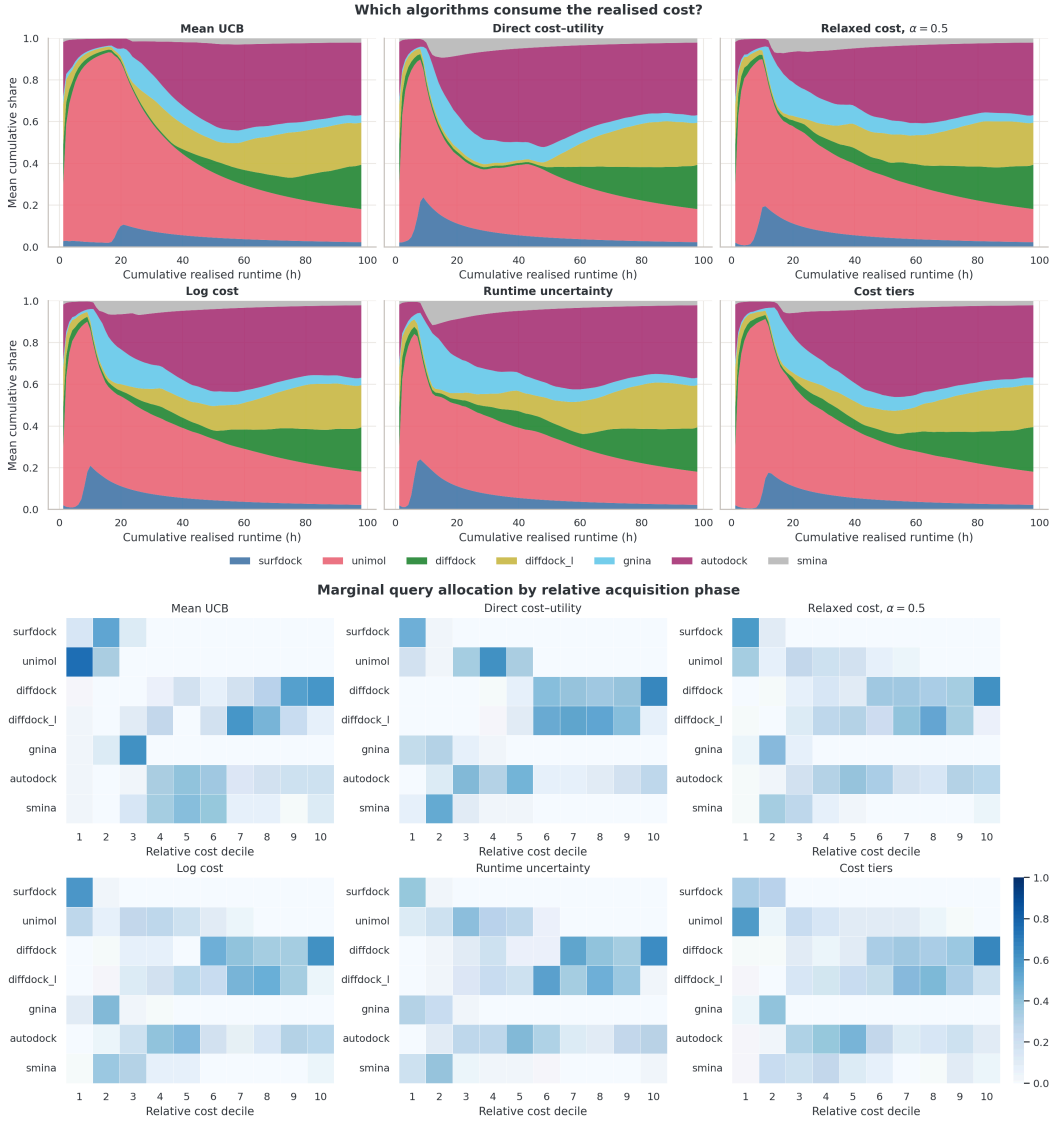


Figure 6: Top: five-fold mean cumulative realized-cost share by docking engine for each acquisition method. Bottom: fold-mean query share in successive 10% fractions of each fold's full realized training-pool cost.

The common allocation effect is concentrated early (Figure 6). Mean-UCB assigns most first-decile queries to Uni-Mol. The cost-aware variants reduce that concentration to different extents and bring SurfDock, GNINA, and Smina forward. Smina's main query share moves from the middle of the Mean-UCB trajectory to approximately the second or third 10%-cost phase. Expensive DiffDock queries are correspondingly deferred. Many cheap Smina queries consume little runtime, while fewer Uni-Mol or extreme AutoDock Vina queries can dominate realized cost. Since every policy eventually observes the same full matrix, these differences describe sequencing.

Signed prediction errors alone do not explain this sequencing. Smina is queried earlier despite early overprediction, whereas Uni-Mol is queried less despite underprediction. This is consistent with coarse between-engine cost separation influencing allocation despite imperfect absolute predictions; it does not show that calibration errors are harmless or establish their causal effect on selector quality.

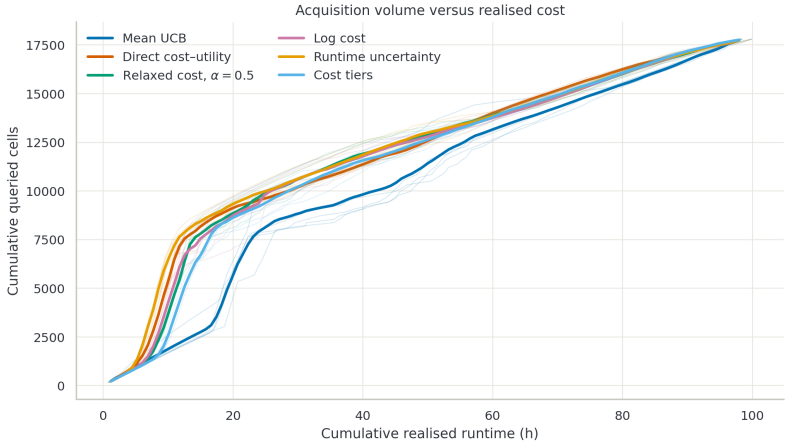


Figure 7: Cumulative queried cells versus realized runtime for AL seed 0. Thick curves are five-fold macro means after interpolation over common realized-cost support; faint curves show observed fold trajectories.

Figure 7 shows greater acquisition volume for the evaluated cost-aware policies over much of the plotted intermediate-cost range, before convergence when the matrix is exhausted. More acquired cells per realized hour do not by themselves establish greater information value or better downstream decisions.

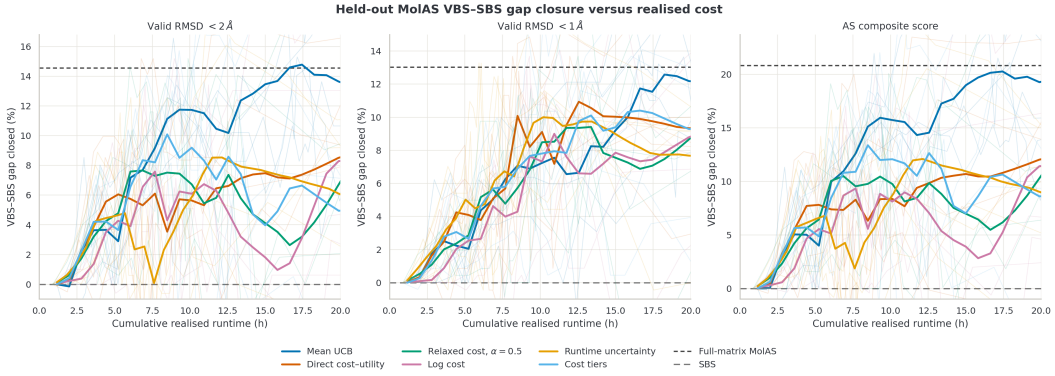


Figure 8: Held-out SBS–VBS gap closure over the first 20 realized hours. Thick curves are interpolated five-fold macro means; faint curves show fold trajectories. Dashed lines denote SBS (0% gap closure) and full-matrix MolAS.

Figure 8 shows formulation-, endpoint-, and budget-dependent downstream trajectories. Some cost-aware policies exceed the Mean-UCB macro curve over portions of the early-budget range, but these descriptive differences do not establish a statistically significant advantage. Interpolation summarizes

evaluated checkpoints; it does not represent a separately trained selector at every plotted runtime. Shared algorithm-level reallocations do not map one-to-one to held-out performance, motivating evaluation of the complete acquisition–completion–selection pipeline.

A.3 Scope, interpretation, and next steps

Our contribution is an empirical evaluation using established active-learning and matrix-completion methods. The findings concern the evaluated Mean-UCB formulations. Random-cell and pure-uncertainty controls are absent, so improvement over random partial acquisition remains unestablished. Broader comparisons should include joint batch acquisition such as BatchBALD, Pareto-aware policies, and budgeted lookahead. BatchBALD requires joint probabilistic information calculations; SVD disagreement alone is insufficient. The current policies are myopic. These comparisons remain next steps.

MolAS learns from completed targets. An observed-entry-only comparison on identical acquisition histories, using a suitable masked objective, would assess dependence on imputation without treating missing outcomes as failures. Adaptive sampling would still make observed entries selective. Feature-conditioned completion models would test dependence on SVD. Additional acquisition and selector seeds, paired comparisons at prespecified realized-cost checkpoints, and appropriately qualified uncertainty estimates would strengthen statistical characterization. Current fold curves are descriptive, without confidence intervals or formal early-budget significance tests; overlapping training sets and repeated checkpoints are not independent replications. Sensitivity to initialization, β, SVD ranks, reranking interval, and clipping remains unestablished. These settings define the evaluated configuration rather than validated deployment defaults.

The acquisition feature cache was normalized over all 3,179 complexes before projection, rather than separately within each outer training fold. This feature preprocessing uses held-out covariates but not their docking outcomes or runtimes; runtime-model fitting and its scaler use only observed training-pool cells. Fold-local feature preprocessing remains untested.

The runtime diagnostics document engine-specific bias on acquired cells, without establishing calibration over the unqueried pool or validating a correction. Calibration using only previously revealed outcomes is a next step, assessed with predictions recorded before each outcome is revealed. Its effects on acquisition and downstream quality require separate evaluation. Guidance on endpoint/budget selection, validation, and monitoring should not be interpreted as an empirically validated stopping or calibration procedure.

Additional datasets, protein–ligand domains, and portfolios would test external validity. Prospective docking is needed to assess hardware transfer, runtime drift, failures, timeouts, retries, and parallel execution. Historical replay does not establish these operational properties, and pose-quality endpoints do not establish experimental hit-discovery benefits. Conclusions remain restricted to the reported retrospective evidence.